

Tutorial

Software Reuse

definition
reason
challenge
difference

1. Suggest why the savings in cost from reusing existing software are not simply proportional to the size of the components that are reused.

The larger part of the software is being reused, the more work has to be done to understand and test the code, more change may be required to accommodate the reuse software.

2. What are the differences between SPL and COTS?

Answer

1. SPL is a set of software products with common core components. Each software in the family specialized in certain aspect.
 2. The major different is the fact that COTS is a single software but SPL is a family of software.
 3. COTS is usually a more generic software. SPL on the other hand is more focused on a specific domain of industry.
 4. COTS is a software readily available in the market for immediate use. In this aspect, SPL may not be as readily usable as COTS.
 5. COTS is rather limited in term of customization as compared to SPL. SPL provide greater degree of customization.
 6. In term of reusability, COTS is a generic software readily reusable by broad range of users while SPL requires certain degree of customization and composition using a range of reusable components.
3. Testing process for COTS-solution system should focus on the interactions between the operational processes and the system configuration. Justify.

Answer

The COTS systems are production-ready systems. Unit tests are completed long before it is available in the market for sales, and hence the user do not need to worry about the technical aspect of the software. The concern of the user, when implementing the COTS system should be focusing on the interactions between the operational processes and the system configuration. A COTS system is useful only if its processes is compatible to the user's operations, and it if doesn't it should be able to be reconfigured.

4. Imagine you're building a dynamic web app—say, a live music playlist editor—using a framework like React or Django. Your app needs to respond to user actions and lifecycle events. Dive into the roles of callbacks and hooks: how do they work in a web framework, and what sets them apart in bringing your app to life?

ANS:

Callbacks are functions you write and pass to the framework to be executed at specific moments, usually triggered by user actions or external events. For example, in your playlist editor, you might pass a callback to a “Play” button’s onClick event in React. When the user clicks, the framework calls your function to start the music—think of it as handing the framework a note saying, “Hey, run this when the user does that!” In JavaScript frameworks, callbacks were traditionally used for asynchronous tasks (e.g., fetching song data), though they’ve largely been replaced by promises since ES2017. They’re about *you telling the framework what to do later*.

Hooks, on the other hand, are special functions provided by the framework that let you “hook into” its lifecycle or state system, giving you control at predefined points without taking over the flow. In React, for instance, the useEffect hook could watch your playlist’s song list and automatically update the UI whenever a new song is added—no manual trigger needed. It’s like planting a sensor in the app that says, “I’ll run my code when this happens.” Hooks are built into the framework’s design (e.g., lifecycle hooks in Angular or React’s state hooks), offering a structured way to tap into its rhythm.

Key Differences:

- Callbacks are your instructions passed to the framework; hooks are the framework’s tools you use.
 - Callbacks run when you say (e.g., on a click); hooks run when the framework decides (e.g., on state change).
5. A software development team is tasked with creating a fitness tracking application that supports multiple user roles (athletes, coaches, health professionals), includes workout planning features, and integrates with external wearable devices (e.g., Fitbit). The client demands delivery within five months and prioritizes scalability to handle growing user numbers, and maintainability.

Complete the following structured short-answer questions based on the scenario:

- Name **one** software reuse technique suitable for this project.
Answer: Application Frameworks
- State **two reasons** why the technique you chose in part 1) is suitable for this project.
Answer:
Frameworks support scalability by offering extensible architectures, allowing the app to handle increasing users as wearable adoption grows. Projects develop with framework are more maintainable because developers extend the framework mainly by inheritance, callbacks, and hooks.

Application Frameworks provide pre-built modules for user management and device integration, enabling the team to deliver the fitness tracking app within the five-month timeline

7. You are a software engineering consultant hired by a company that develops mobile applications for different industries, such as healthcare, finance, and education. The company is considering adopting software reuse techniques to reduce development time and costs while maintaining high quality. They are particularly interested in understanding the differences between Application Frameworks and Software Product Lines (SPL).

Scenario:

The company plans to develop a new set of mobile apps for small businesses, including:

- A point-of-sale (POS) app for retail stores.
- An appointment scheduling app for service-based businesses.
- An inventory management app for warehouses.

All apps need to run on both iOS and Android, and they must share common features like user authentication, payment processing, and data analytics.

Tasks:

- Define **Application Frameworks** and **Software Product Lines (SPL)**, highlighting their key differences.

Answer: An Application Framework is a pre-built, extensible structure (e.g., a set of reusable classes or modules) that developers customize for specific applications, such as React Native for mobile apps, which provides reusable components like navigation and UI elements. In this case, a framework could be used to build each app by extending a common base with platform-specific features for iOS and Android. In contrast, a **Software Product Line (SPL)** is a family of related software products sharing a common core, designed for reuse across variants, such as a base app with shared features (authentication, payments) specialized into POS, scheduling, and inventory versions. The key difference is that frameworks focus on providing a technical foundation for individual apps, while SPLs emphasize a strategic, domain-specific family of products with reusable components tailored to varying requirements.

- Recommend which technique (Application Frameworks or SPL) is more suitable for the company's needs. Justify your choice with at least three specific reasons based on the scenario.

Answer: I recommend using a Software Product Line (SPL) for this project. First, SPLs allow the company to create a single core system with shared features (e.g., authentication, payment processing, analytics, customer management, product management, etc) that can be reused

across all three apps, reducing redundant development effort and ensuring consistency. Second, the specialization capability of SPLs (e.g., functional specialization for POS vs. scheduling) aligns perfectly with the need for distinct yet related apps, enabling efficient customization for each business type. Third, SPLs support cross-platform deployment (iOS and Android) through platform specialization, ensuring the apps work seamlessly on both operating systems while leveraging the same reusable core, which meets the tight deadline and cost-efficiency goals.

- Identify two potential challenges the company might face when implementing your recommended technique.

Answer:

- One challenge of using an SPL is the **high initial investment** in designing the common core and defining reusable components, which could strain resources under a tight deadline.
- A second challenge is **maintaining consistency across variants** as the apps evolve, as changes to the core could break specialized features.